Scikit Learn Implementation

Scikit Learn

Scikit-learn is an open-source Python library used for machine learning and data analysis. It provides simple and efficient tools for tasks like classification, regression, and clustering. Built on libraries like NumPy and SciPy, it is easy to use and widely adopted. It is popular among both beginners and professionals for developing predictive models.

Step 1: Install Scikit-Learn

First we need to install our dependency, the Python library scikit-learn(sklearn) which is a free software machine learning library for the Python Programming Language. The random library is part of the python standard libraries so we don't need to install it, it's already available for us

- pip install scikit-learn

Step 2: Create Your Training dataset

Usually if you have a data set, you would want to split it into a training set, validation set, and testing set. Since this is a simple example we are creating our own training set to train our linear regression model with.The training set will contain the input and expected output values of the data set that I want to train the model on. The input will be random integer values created by the library Random we will use the method randint(a,b) , which returns a random integer N such that a <= N <= b. The output will be determined by a function that we will create.

Recall a function usually called 'f' takes in some input/parameter value usually called 'x' and produces some output value usually represented by 'y', such that f(x) = y. The function that I will create will take in multiple parameters so not just 'x' but 'a', 'b', and 'c' instead such that we get the following function.

f(a, b, c) = 7a + 3b + 4c+9d= y

Example of how this function works, if we had values a=1, b=2, c=20, d=30 then y would equal 363 like the following:

f(1, 2, 20, 30) = 7(1) + 3(2) + 4(20) + 9(30) = 363

Example of Training Set:

Input				Output
a	**b**	**c**	**d**	**y**
1	2	20	30	363
4	3	8	2	87
6	1	5	9	145
5	2	7	4	105

Once we have created the training set, we will split each row into two lists (a input training set , and it's corresponding output training set). So now we have two lists of all the inputs and their corresponding outputs.

Code to Generate Training Set:

```
#Import the libraries
from random import randint
from sklearn.linear_model import LinearRegression

# Create a range limit for random numbers and a count of the number of rows
train_set_limit=1000
train_set_count=100

# Create an empty list of the input set 'X' and output for each training set 'Y'
train_input=list()
train_output=list()

##Create and append a randomly generated data set to the input and output
for i in range(train_set_count):
    a= randint(0, train_set_limit)
    b= randint(0, train_set_limit)
    c= randint(0, train_set_limit)
    d= randint(0, train_set_limit)
    #Create a linear function for the output dataset 'Y'
    op=(7*a)+(3*b)+(4*c)+(9*d)
    train_input.append ([a,b,c,d])
    train_output.append(op)
```

Step 3: Train Our Linear Regression Model

Next we will use the method LinearRegression() from the Python library scikit-learn to train and create our model. This model will try to “model” the function that we created for the training dataset.

Note: When we ‘fit’ a function that is just another word for training.

```
#Create a linear regression object
predictor=LinearRegression(n_jobs=-1)
#fit the linear model
predictor.fit(X=train_input, y=train_output)
```

Step 4 : Test Our Linear Regression Model

Now let’s see if our LinearRegression function can approximate the function that we created and give us an accurate prediction (the correct answer).

X = [[10, 20, 30, 40]]

The outcome should be 7*10 + 3*20 + 4*30 + 9*40 = 610. Let’s see what we got…

```
#Create our testing data set
X_test= [[10,20,30,40]]

# Predict the ouput of the test data
outcome=predictor.predict(X=X_test)

#The estimated coefficients for the linear regression problem.
coefficients= predictor.coef_
```

```
print("the outcome of the dataset is:\n")
print('Outcome:{}\nCoefficients:{}'.format(outcome,coefficients))
```

The result of the above code is:

```
the outcome of the dataset is:
Outcome:[610.]
Coefficients:[7. 3. 4. 9.]
```